CAMPUSLINK SYSTEM ARCHITECTURE & BLUEPRINT

CAMPUSLINK

System Architecture, Operational Mechanics & Complete Technology Reference Guide

An In-Depth Engineering Blueprint of *How Things Work* and *What Things Have Been Used for What*

Target Platform:	CAMPUSLINK Career Intelligence Operating System
Architecture Pattern:	Local-First Monorepo (Express/Prisma/PostgreSQL + React/Vite/Tailwind)
Core Philosophy:	Zero Black-Box AI · Fully Deterministic Rubrics & Matching · Cryptographic Provenance
Primary User Personas:	Students, Placement Officers (Staff), Corporate Recruiters, System Administrators
Database Engine:	PostgreSQL Relational DB with Prisma 5 Schema Migrations & Relations
Security & Auth:	scrypt Hashing, CSPRNG HMAC-SHA256 Email OTP, Revocable HttpOnly Sessions, Strict CORS & CSRF
Documentation Scope:	Full System Mechanics, Algorithms, Component Roles, Database Schema & Operational Runbook

ARCHITECTURAL PRINCIPLE: TRANSPARENT & EVIDENCE-BACKED PLACEMENT

Unlike platforms that rely on opaque black-box AI matchmakers or unverified resume claims, CAMPUSLINK operates entirely on explainable mathematical models, publicly auditable grading rubrics, and verifiable project evidence. Every metric, skill level advancement, and job compatibility factor is backed by persisted relational records in PostgreSQL.

CONFIDENTIAL & COMPREHENSIVE ENGINEERING REFERENCE
Published for Development, Architecture Review & Auditing

1. Executive Summary & Core System Philosophy

CAMPUSLINK is a modern, high-assurance career intelligence and campus placement operating system. It replaces manual placement tracking, unverified resume claims, and disconnected spreadsheets with a unified, evidence-backed ecosystem connecting Students, Placement Cell Officers (Staff), and Corporate Recruiters.

Modern hiring suffers from a profound credibility crisis: resumes contain exaggerated, self-reported skills, while companies rely on unpredictable machine learning classifiers that candidates cannot inspect or trust. CAMPUSLINK was engineered from first principles around three inviolable architectural commitments:

• **1. Zero Black-Box AI Guarantee:** Every score in the system—whether job matching compatibility, sprint assessment grading, interview performance, or student placement readiness—is computed using a transparent, published, deterministic rule-based formula. The system never delegates evaluation to unexplainable neural networks or generative LLMs.

• **2. Provenance-Based Credentialing:** The platform strictly delineates between *Self-Reported* claims (which carry minimal weight) and *Evidenced* credentials (backed by verified project repositories and rubric-evaluated Sprint submissions). Unverified claims earn zero evidence points.

• **3. Local-First & Defense-in-Depth Security:** Designed to operate independently on local infrastructure without cloud vendor lock-in. Loopback-only bindings, HttpOnly session cookies, cryptographic scrypt password hashing, CSPRNG HMAC-SHA256 OTP verification, and server-side RBAC ensure enterprise-grade security.

LAYER	TECHNOLOGY & COMPONENTS	PRIMARY RESPONSIBILITY
Client Layer (Frontend)	React 19, Vite 8, React Router v7, TanStack Query, Tailwind CSS v4, Lucide React	Single-Page Application (SPA) providing responsive workspaces: Career Twin, Opportunity Mirror, Skill Sprint, Interview Arena, Applications, Placement Tower.
API & Controller Layer (Backend)	Express 4, TypeScript 5, Zod, Helmet, CORS, Express-Rate-Limit	RESTful endpoints with strict JSON schemas, IP rate-limiting budgets, CSRF origin verification, and server-side role-based access control (RBAC).
Core Business Logic & Engines	Deterministic Matching Engine, Rubric v1 Evaluator, Arena Bank Evaluator, Readiness Calculator	Calculates explainable fit scores, evaluates code sprint submissions, tallies STAR interview answers, and compiles holistic student readiness.
Data Access & Storage Layer	Prisma ORM 5, PostgreSQL 16+	Relational schema management with forward-only migrations, type-safe transactions, unique constraints, and foreign key cascades.
Cryptographic & Messaging Layer	Node.js Crypto (scrypt, HMAC-SHA256, CSPRNG), Nodemailer SMTP	Issues rejection-sampled 6-digit OTPs, hashes secrets with salts and peppers, stores SHA-256 session tokens, and sends transactional verification emails.

2. What Things Have Been Used For What? (Technology Stack Breakdown)

Every single package, library, and framework in CAMPUSLINK has been deliberately selected to fulfill a precise architectural need without introducing bloat or black-box dependencies. Below is the comprehensive audit of all technologies utilized across the backend and frontend codebases.

2.1 Backend Technologies & Runtime Dependencies

PACKAGE / TOOL	VERSION	EXACT ROLE IN CAMPUSLINK	WHY IT WAS CHOSEN
Express	`4.21.2`	HTTP REST API web server. Powers all API routes (`/api/auth`, `/api/students`, `/api/opportunities`, etc.), JSON body parsing, and middleware chains.	Lightweight, battle-tested, unopinionated, and provides robust control over request pipelines, cookies, and error filters.
TypeScript	`5.6.3`	Static type system across the entire backend. Defines strict interfaces for matching inputs, API response contracts, DB queries, and middleware signatures.	Eliminates runtime type errors, guarantees end-to-end data integrity, and synchronizes perfectly with Prisma-generated types.
Prisma ORM (`@prisma/client`)	`5.22.0`	Data access layer, schema definition (`schema.prisma`), forward-only migration engine, and multi-table atomic transaction manager (`prisma.$transaction`).	Provides compile-time type-safe database queries, auto-generated TypeScript models, transparent relation joining, and declarative migrations.
PostgreSQL	`16+`	Primary relational database storing users, sessions, student profiles, skills, project evidence, jobs, applications, sprint attempts, and announcements.	Enterprise ACID compliance, robust indexing (B-trees on foreign keys and emails), JSON column querying for hiring criteria, and relational cascade integrity.
Zod	`3.23.8`	Runtime payload validation and parsing. Enforces strict request schemas (`.strict()`) on registration, job creation, skill updates, and reviews.	Strips or rejects unauthorized client fields, preventing parameter pollution and role escalation (e.g. students attempting to pass `role: 'ADMIN'`).
scrypt (Node crypto)	`Built-in`	Cryptographic password hashing algorithm for user authentication. Formats hashes as `scrypt$v1$N=16384$r=8$p=1$<salt>$<hash>`.	Memory-hard key derivation function specifically designed to resist hardware-accelerated GPU, ASIC, and brute-force cracking attacks.
HMAC-SHA256 & CSPRNG	`Built-in`	Generates uniform 6-digit OTPs via `randomInt(0, 1000000)` and hashes them with an application pepper before storing in `RegistrationAttempt`.	Eliminates modulo bias in random number generation. Even if the database is leaked, raw OTP codes cannot be brute-forced without the server pepper.
Nodemailer	`10.1.0`	SMTP mail client used in the student registration workflow (`POST /api/auth/register/start`) to deliver live 6-digit verification codes to applicant inboxes.	Reliable standard for Node.js email transport with direct TLS/SSL support, app-password authentication, and fail-safe error handling.
Helmet	`8.0.0`	Sets critical HTTP security response headers (`X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, `X-DNS-Prefetch-Control`).	Mitigates common web vulnerabilities like cross-site scripting (XSS), clickjacking, and MIME-sniffing attacks.
CORS	`2.8.5`	Restricts cross-origin resource sharing strictly to the configured loopback origin (`http://127.0.0.1:5173`) with explicit credential support.	Dynamic function evaluation ensures unauthorized origins receive zero Access-Control-Allow-Origin headers, blocking external browser reads.
Express-Rate-Limit	`7.4.1`	IP-based rate-limiting middleware enforcing strict request budgets: Global (300/15m), Login (20/15m), Register Start (20/15m), OTP Verify (60/15m).	Prevents Denial of Service (DoS), automated OTP enumeration, brute-force password guessing, and registration flooding.
tsx	`4.19.2`	Development runtime executing TypeScript directly via esbuild (`tsx watch src/index.ts`, `tsx prisma/seed.ts`).	Instant startup, high-speed execution, seamless TypeScript module resolution, and file watching without manual compilation steps.
Vitest	`2.1.8`	Automated test runner executing comprehensive integration tests for auth, CSRF, matching algorithms, rubrics, and application workflows.	Blazing fast execution, full ESM/TypeScript support, and complete compatibility with standard test assertions.

2.2 Frontend Technologies & Client Dependencies

PACKAGE / TOOL	VERSION	EXACT ROLE IN CAMPUSLINK	WHY IT WAS CHOSEN
React	`19.2.8`	Core declarative UI component library powering all workspaces, reactive forms, interactive modals, and data dashboards.	Component-based architecture, highly efficient reconciliation, robust hook primitives (`useState`, `useEffect`, `useContext`), and state isolation.
Vite	`8.3.0`	Next-generation frontend dev server and production bundler configured with `strictPort: true` on loopback `127.0.0.1:5173`.	Sub-second Hot Module Replacement (HMR), lightning-fast esbuild pre-bundling, optimized Rollup production builds, and strict port guarding.
React Router (`react-router-dom`)	`7.18.4`	Client-side Single Page Application (SPA) routing. Manages public routes (`/`, `/login`, `/register`) and protected routes wrapped in `WorkspaceLayout`.	Clean declarative URL mapping, seamless navigation without full page reloads, query string management, and integration with authentication redirect gates.
TanStack Query (`@tanstack/react-query`)	`5.104.1`	Server-state management, asynchronous data fetching, client-side caching (`staleTime: 15s`), and automatic cache invalidation on mutations.	Eliminates manual `useEffect` fetch boilerplate, manages loading/error states automatically, prevents duplicate network calls, and synchronizes server state.
Tailwind CSS v4 (`@tailwindcss/vite`)	`4.3.3`	Utility-first styling framework with custom design tokens (`--color-signal`, `--color-ink`, `--color-brand`), glassmorphism effects, and responsive grids.	Next-generation zero-config Vite integration, instant CSS generation, design token consistency, and clean modern dark-mode aesthetic.
Lucide React	`1.55.0`	Modern, lightweight SVG icon system utilized across navigation bars, metric indicators, status badges, and action buttons.	Clean aesthetic consistency, zero external font downloads, tree-shakeable SVG components, and customizable dimensions and strokes.
Oxlint	`1.81.0`	High-performance Rust-based linter used to verify code quality, syntax standards, and React best practices during builds.	Executes 50-100x faster than traditional ESLint, catching bugs, bad imports, and type inconsistencies in milliseconds.

2.3 Project Directory & File Manifest

FILE OR DIRECTORY	PRIMARY ARCHITECTURAL ROLE & FUNCTION
`backend/src/app.ts`	Express application factory. Configures Helmet, strict loopback CORS, JSON limits, CSRF origin checks, rate-limiters, and mounts all API routers.
`backend/src/index.ts`	Server entrypoint. Binds Express strictly to `127.0.0.1:4000`, wires graceful shutdown handlers (`SIGTERM`, `SIGINT`), and disconnects Prisma cleanly.
`backend/src/lib/auth.ts`	Authentication middleware. Defines `requireAuth`, `requireRole`, cookie serialization/clearing, and request user context population.
`backend/src/lib/otp.ts`	CSPRNG one-time code generator (`randomInt`), peppered HMAC-SHA256 hasher, and constant-time string comparison (`timingSafeEqual`).
`backend/src/lib/match.ts`	Explainable opportunity matching engine. Computes fit score, met requirements, gaps, bonuses, and step-by-step guidance without AI.
`backend/src/lib/rubric.ts`	Deterministic Sprint grading engine (`rubric-v1`). Scores repository link, completion, write-up depth, and engineering tradeoffs.
`backend/src/lib/arena.ts`	Interview Arena practice evaluator (`arena-bank-v1`). Grades STAR structure, substance, quantification, and assigns practice points.
`backend/src/lib/sprint-evidence.ts`	Transactional evidence upsert utility. Reuses existing project rows for repeated passing attempts to prevent duplicate evidence inflation.
`backend/src/routes/register.ts`	Public student onboarding endpoints: Step 1 (`/register/start`) emails code; Step 2 (`/register/verify`) verifies OTP; Step 3 (`/register/complete`) creates account.
`frontend/src/main.tsx`	React entrypoint and router configuration. Maps public landing page, login, register, and protected workspace routes inside `WorkspaceLayout`.
`frontend/src/store/session.tsx`	IdentityGate context provider. Probes `GET /api/auth/me`, handles 401 redirects to `/login`, and supplies user role and profile ID globally.
`frontend/src/pages/CareerTwin.tsx`	Student profile and skill management UI. Visualizes self-reported vs evidenced skills, project links, and personal academic details.
`frontend/src/pages/OpportunityMirror.tsx`	Interactive job matching dashboard. Displays transparent fit percentages, met criteria, missing gaps, and one-click application submission.
`frontend/src/pages/SkillSprint.tsx`	Practical technical assessment challenge UI. Provides task checklist, repo submission inputs, and displays real-time rubric feedback.
`frontend/src/pages/InterviewArena.tsx`	Mock interview practice room. Simulates Behavioral, Technical, and Situational questions with instant rule-based scoring feedback.
`frontend/src/pages/PlacementTower.tsx`	Placement officer analytics dashboard. Displays student cohort readiness, application pipeline metrics, and the interactive What-If Lab.
`frontend/src/pages/Passport.tsx`	Verifiable student credential passport. Displays skill provenance, linked project proofs, rubric evaluations, and holistic readiness score.

3. How Things Work: Deep Technical Mechanics & Workflows

This section deconstructs the operational workflows of CAMPUSLINK step by step, illustrating the exact sequence of events, database mutations, mathematical formulas, and security checks governing every feature.

3.1 Public Student Onboarding & Session Security

Student registration is public and self-service, but fortified with an uncompromising 3-step email verification protocol. Crucially, **no User or StudentProfile record is created until the student's email address is cryptographically proven**.

Step 1: Initiation (`POST /api/auth/register/start`)

The student enters their Full Name, 8-character Enrollment Number (e.g., `21BFH987`), Branch, Graduation Year, College, and Email. The server strictly validates all inputs with Zod, checks that neither the email nor enrollment number already exists in `User` or `RegistrationAttempt`, generates an un-biased 6-digit OTP using `crypto.randomInt()`, hashes it with HMAC-SHA256 using an application pepper, persists a `RegistrationAttempt` record, and dispatches the code via Nodemailer SMTP. A 60-second resend cooldown and 10-minute expiry are enforced.

Step 2: Verification (`POST /api/auth/register/verify`)

The student inputs the 6-digit code. The server performs a constant-time comparison (`timingSafeEqual`) against the stored HMAC hash. If correct, the OTP hash is immediately cleared (single-use), `verifiedAt` is recorded, and a high-entropy 256-bit `completionToken` valid for 20 minutes is returned in memory. If incorrect, an attempt counter increments (capped at 5 wrong attempts before permanent invalidation).

Step 3: Completion (`POST /api/auth/register/complete`)

The student submits their chosen password along with the `completionToken`. The password must satisfy the security policy (12–256 chars, letter + number). In a single atomic database transaction, the server: (a) verifies the completion token hash, (b) marks the attempt as consumed, (c) hashes the password with scrypt, (d) creates the `User` row with role forced to `STUDENT`, (e) creates the linked `StudentProfile` copying verified data from the attempt row, and (f) generates a 7-day session token, setting the `campuslink_session` HttpOnly cookie. The user is logged in immediately.

SESSION SECURITY ARCHITECTURE: HASH-ONLY SERVER STORAGE

Raw session tokens exist solely inside the client's HttpOnly cookie (`SameSite=Lax`, `Max-Age=7d`). The database stores exclusively `SHA-256(token)` inside the `Session` table. In the event of a database compromise, attackers cannot forge or hijack active user sessions. Revoking a session (`POST /api/auth/logout`) sets `revokedAt = now()`, instantly neutralizing the cookie across all subsequent requests.

3.2 Career Twin & Provenance-Based Credentialing

The Career Twin (`/twin`) acts as the student's living digital resume and skill inventory. Unlike standard job boards where candidates claim 'expert' status with zero proof, CAMPUSLINK institutes strict provenance tracking across 4 standardized proficiency tiers:

LEVEL	RANK	PROVENANCE & MEANING	EVIDENCE CRITERIA
BEGINNER	1	Foundational theoretical understanding; basic familiarity with syntax and concepts.	Self-reported claim; awarded automatically when adding skills or unevidenced projects.
INTERMEDIATE	2	Working operational competence; able to build features independently.	Requires either linked project evidence or scoring 70+ on a beginner Sprint assessment.
ADVANCED	3	In-depth architectural capability; understands tradeoffs, performance, and internal patterns.	Earned by passing complex Sprint assessments and linking multi-skill repositories.

EXPERT	4	Mastery; capable of debugging framework internals, designing systems, and leading architecture.	Highest echelon; reserved for verified multi-sprint achievements and verified code reviews.

3.3 Opportunity Mirror: Explainable Matching Engine

The Opportunity Mirror (`/mirror`) compares a student's verified skills against open corporate job criteria. Instead of delivering an opaque match percentage from a machine learning black-box, the match score is derived from a deterministic, published mathematical algorithm (`MATCH_METHOD`).

MATHEMATICAL FORMULA: OPPORTUNITY MATCHING ALGORITHM (lib/match.ts)

```
Weight = Rank(minLevel) x 2
Points = FullWeight + min(linkedProjects, 2) x 0.5 [if student level >= minLevel]
Points = Weight x 0.75 [if student level == minLevel - 1 (Partial Credit)]
Points = 0 [if student level < minLevel - 1]
Bonus = +1 point [for each nice-to-have skill at INTERMEDIATE+]
Score = min(100, round(100 x (Points_Earned / Max_Possible_Points)))
```

Every job posting specifies requiredSkills with minimum levels and niceToHave skills. The response explicitly returns the formula method, met criteria, exact skill gaps, bonus points earned, and a tailored nextStep recommendation (e.g. 'Close React gap with a targeted sprint to lift this match').

3.4 Skill Sprint & Transparent Rubric Evaluation

Skill Sprints (`/sprint`) provide hands-on coding and architecture challenges tied to specific target skills. When a student submits their solution (checklist tasks completed, Git repository URL, and engineering write-up), the backend evaluates the submission against a strict rule-based rubric (`rubric-v1`).

MATHEMATICAL FORMULA: SPRINT GRADING RUBRIC v1 (lib/rubric.ts)

```
Score = 40 (Base Points)
+ 25 (if all checklist tasks attested)
+ 15 (if valid verifiable HTTP/HTTPS repo or deployment link provided)
+ 15 (if write-up depth >= 40 words explaining tradeoffs; +8 if >= 15 words)
+ 5 (bonus: if write-up explicitly names quality attributes: tests, a11y, perf, tradeoffs)
Total Cap: 100 points | Promotion Threshold: 70 points
```

When a submission achieves a score of 70 or higher, the backend automatically performs a dual transactional promotion: (1) The rewarded skill is upgraded one level in StudentSkill, and (2) An evidence project named 'Sprint: <Title>' is upserted in the Project table, establishing permanent verifiable proof.

3.5 Interview Arena & Practice Points Ledger

The Interview Arena (`/arena`) prepares students for rigorous placement interviews across three core categories: **Behavioral** (leadership, conflict resolution), **Technical** (rendering, concurrency, algorithms), and **Situational** (deadline triage, stakeholder pressure).

MATHEMATICAL FORMULA: ARENA ANSWER SCORING & POINTS LEDGER (lib/arena.ts)

```
Score = 35 (Base Points)
+ 30 (Substantive depth: >= 80 words; +18 if >= 40 words)
+ 15 (Quantification: contains explicit numerical metric or impact data)
+ 10 (Reasoning keywords: 'because', 'tradeoff', 'therefore', 'however')
+ 10 (STAR structure keywords: 'situation', 'task', 'action', 'result')
Practice Points = floor(Session_Score / 10) [Range: 0 to 10 points]
```

Practice points are recorded in a dedicated ArenaPointLedger table keyed uniquely by sessionId. If a student re-evaluates or edits their answer, the ledger row updates in-place, mathematically preventing point duplication or gaming.

3.6 Application Lifecycle & State Machine Transitions

Job applications follow a strictly guarded Finite State Machine (FSM). Transition permissions are strictly enforced based on the actor's session role to prevent unauthorized status tampering.

CURRENT STATUS	PERMITTED NEXT STATUS	AUTHORIZED ROLES	BUSINESS RULES & BEHAVIOR
APPLIED	SHORTLISTED, REJECTED, WITHDRAWN	Student (WITHDRAWN only); Recruiter & Staff (SHORTLISTED, REJECTED)	Initial state upon student application. Unique constraint on (jobId, studentProfileId) prevents duplicate submissions.
SHORTLISTED	HIRED, REJECTED, WITHDRAWN	Placement Staff only (HIRED); Recruiter & Staff (REJECTED); Student (WITHDRAWN)	Candidate has cleared preliminary screening. Recruiters can only shortlist for their own company's jobs.
REJECTED	*None (Terminal)*	N/A	Terminal state. Application cannot transition backward to APPLIED or SHORTLISTED.
HIRED	*None (Terminal)*	N/A	Terminal state. Only Placement Officers (STAFF/ADMIN) can confirm an application as HIRED.
WITHDRAWN	*None (Terminal)*	N/A	Terminal state. Student initiated withdrawal from consideration.

3.7 Placement Tower, Cohort Analytics & What-If Lab

The Placement Tower (`/tower`) provides placement officers with macro-level cohort intelligence. It features live stats: registered students, active jobs, pipeline stage counts, and cohort readiness. It also incorporates the **What-If Lab**: a client-side simulation engine that models how targeted cohort interventions (e.g., closing 2 skill gaps per student or running targeted sprints) will mathematically impact hiring readiness percentages.

MATHEMATICAL FORMULA: HOLISTIC STUDENT READINESS SCORE (lib/match.ts & /passport)

```
Readiness Score = Proficiency_Factor (50%) + Evidence_Coverage (30%) + Activity_Momentum (20%)
Proficiency = (Mean_Skill_Rank / 4.0) x 50
Evidence = (Count_of_Evidenced_Skills / Total_Skills_Count) x 30
Momentum = min(20, (Applications_Filed x 4) + (Evaluated_Sprints x 6))
Total Scale: 0 to 100
```

A student with 10 claimed skills but zero linked projects or sprints will receive zero points from the evidence pillar. The formula directly incentivizes practical code verification and proactive application engagement.

3.8 Targeted Announcements & Materialized Delivery Pattern

When placement officers publish announcements (`POST /api/announcements`), the backend resolves the target audience (ALL, APPLIED, SHORTLISTED, INTERVIEW_SELECTED) and executes the **Materialized Recipient Pattern**:

Instead of querying student states dynamically on every page load, the system resolves the matching students at publication time and creates permanent `AnnouncementRecipient` records in a single database transaction. This guarantees that: (1) The audience snapshot is permanently frozen as of broadcast time, (2) Students who register or change status later do not receive historical notices meant for past cohorts, and (3) Read receipts (`readAt`) are tracked individually per student.

---

4. Database Architecture & Relational Data Models

CAMPUSLINK utilizes PostgreSQL managed via Prisma ORM. The relational schema is structured into distinct functional domains: Identity & Sessions, Skills & Evidence, Recruitment & Jobs, Assessments, and Messaging.

DOMAIN & MODEL	PRIMARY ATTRIBUTES & FIELDS	FOREIGN KEYS & RELATIONS	CONSTRAINTS & INDEXES
Identity `User`	id (cuid), email, passwordHash, role (STUDENT, STAFF, RECRUITER, ADMIN), isActive, companyId	1:1 StudentProfile, 1:N Session, 1:N Announcement, N:1 Company	`@unique(email), @@index([role]), @@index([companyId])`
Security `Session`	id, tokenHash, userId, expiresAt, revokedAt, userAgent, ip	N:1 User (onDelete: Cascade)	`@unique(tokenHash), @@index([userId]), @@index([expiresAt])`
Onboarding `RegistrationAttempt`	id, email, activeEmail, fullName, enrollmentNo, branch, graduationYear, otpHash, completionTokenHash	Self-contained signup workflow	`@unique(activeEmail), @@index([email]), @@index([consumedAt])`
Profile `StudentProfile`	id, userId, fullName, enrollmentNo, branch, graduationYear, cgpa, resumeUrl, institution	1:1 User, 1:N StudentSkill, 1:N Project, 1:N Application, 1:N Attempt	`@unique(userId), @unique(enrollmentNo), @@index([graduationYear])`
Skills `StudentSkill`	id, studentProfileId, skillId, level (BEGINNER, INTERMEDIATE, ADVANCED, EXPERT)	N:1 StudentProfile, N:1 Skill	`@@unique([studentProfileId, skillId]), @@index([skillId])`
Evidence `Project`	id, studentProfileId, title, description, url, skills (String[]), assessmentId	N:1 StudentProfile (onDelete: Cascade)	`@@index([studentProfileId]), @@index([studentProfileId, assessmentId])`
Corporate `Company`	id, name, website, industry, createdAt	1:N JobOpportunity, 1:N User	`@unique(name)`
Hiring `JobOpportunity`	id, companyId, title, description, location, type, status, requiredSkills (Json), niceToHave (String[])	N:1 Company (onDelete: Cascade), 1:N Application	`@@index([companyId]), @@index([status]), @@index([type])`
Funnel `Application`	id, jobId, studentProfileId, status (APPLIED, SHORTLISTED, REJECTED, HIRED, WITHDRAWN)	N:1 JobOpportunity, N:1 StudentProfile	`@@unique([jobId, studentProfileId]), @@index([status])`
Assessments `AssessmentAttempt`	id, assessmentId, studentProfileId, score, status, repoUrl, note, feedback, startedAt, submittedAt	N:1 Assessment, N:1 StudentProfile	`@@index([assessmentId]), @@index([studentProfileId])`

Interview `ArenaPointLedger`	id, studentProfileId, sessionId, points, createdAt	1:1 InterviewSession, N:1 StudentProfile	`@unique(sessionId), @@index([studentProfileId])`
Messaging `Announcement`	id, title, message, category, audience, isUrgent, authorId, jobId, companyId, eventAt, venue	N:1 User, 1:N AnnouncementRecipient	`@@index([authorId]), @@index([jobId]), @@index([createdAt])`
Receipts `AnnouncementRecipient`	id, announcementId, studentProfileId, readAt, createdAt	N:1 Announcement (Cascade), N:1 StudentProfile	`@@unique([announcementId, studentProfileId])`

5. Security Architecture & Defensive Engineering

CAMPUSLINK adheres to defense-in-depth principles. Because the application processes student identities, corporate job postings, and evaluation records, the architecture enforces multi-layered controls:

• **Network & Binding Isolation:** The backend binds strictly to `127.0.0.1:4000` (IPv4 loopback), and the Vite frontend binds strictly to `127.0.0.1:5173` with `strictPort: true`. Neither binds to wildcard interfaces (`0.0.0.0`), preventing unintended local area network exposure.

• **Strict Origin CORS & CSRF Enforcement:** CORS evaluates origins dynamically via a callback, returning zero Access-Control headers to non-whitelisted origins. All mutating requests (POST, PATCH, DELETE) pass through `csrfOriginCheck` verifying the `Origin` or `Referer` header against `CORS_ORIGIN`.

• **Server-Side Role-Based Access Control (RBAC):** Authentication is verified exclusively via session cookies. Client requests carrying foreign `studentProfileId` or `companyId` parameters are disregarded; ownership is derived authoritatively from the validated server session.

• **Rate-Limiting Budgets:** Configured via `express-rate-limit`: Global API (300 requests/15 min), Login (20/15 min), Registration Start (20/15 min), and OTP Verification (60/15 min), preventing brute-force attacks and credential stuffing.

• **Sanitized Error Handling:** Error middleware intercepts all exceptions. Stack traces, database queries, and credentials are logged solely on the local server console. Client responses return sanitized messages (e.g. `Internal server error`), preventing reconnaissance leaks.

HTTP CODE	STANDARDIZED MEANING & USAGE IN CAMPUSLINK
200 OK	Successful query or state mutation; returns `{ ok: true, data: ... }`.
201 Created	Successful entity creation (e.g. project submitted, application filed, user completed signup).
400 Bad Request	Zod validation failure with specific field path issues, or malformed JSON syntax.
401 Unauthorized	Missing, expired, revoked, or invalid session cookie; account inactive. Routine trigger for login redirect.
403 Forbidden	Authenticated caller lacks permissions (e.g., student attempting recruiter routes, cross-user edit).
404 Not Found	Requested student, job, assessment, application, or prompt does not exist.
409 Conflict	Duplicate record: duplicate email, existing enrollment number, or duplicate job application.
410 Gone	Expired registration OTP code or expired completion token.
422 Unprocessable	Legal request syntax, illegal business state: applying to closed job, invalid FSM transition, empty audience.
429 Too Many Requests	Rate-limit budget exceeded (e.g. >5 incorrect OTP attempts or resend cooldown active).
500 Internal Error	Sanitized server fault; internal details and stack traces logged server-side only.
503 Unavailable	PostgreSQL database unreachable (`/api/ready`) or SMTP mail configuration missing.

6. Operational Runbook & Local Setup Guide

CAMPUSLINK is architected for immediate local development and deterministic execution on Windows PowerShell.

6.1 Environment Variables Configuration (`backend/.env`)

VARIABLE	SAMPLE VALUE	PURPOSE & DESCRIPTION
`DATABASE_URL`	`postgresql://postgres:pwd@127.0.0.1:5432/campuslink`	PostgreSQL connection string with credentials and database name.
`PORT`	`4000`	Express HTTP server port (Default: 4000).
`HOST`	`127.0.0.1`	Loopback IP interface binding (keep 127.0.0.1 for isolation).
`CORS_ORIGIN`	`http://127.0.0.1:5173`	Permitted client browser origin for CORS and CSRF verification.
`SMTP_HOST`	`smtp.gmail.com`	Outbound SMTP mail server hostname.
`SMTP_PORT`	`465`	SMTP SSL port (465) or TLS port (587).
`SMTP_USER`	`admissions@college.edu`	Authenticated mailbox username / email address.
`SMTP_PASS`	`16-character-app-password`	Google App Password or SMTP account secret (never Git-committed).
`MAIL_FROM`	`"CAMPUSLINK" <admissions@...>`	Sender display name and address for verification emails.
`OTP_PEPPER`	`64-character-hex-secret`	Persistent HMAC key preventing OTP invalidation across process restarts.

6.2 Step-by-Step PowerShell Execution Sequence

Step 1: Configure Local Database & Environment

```
cd backend
.\scripts\Set-LocalEnv.ps1
```

Step 2: Apply Forward-Only Migrations & Verify Connection

```
npx prisma migrate dev --name init
npm run db:check
```

Step 3: Seed Synthetic Baseline Development Dataset

```
npm run prisma:seed
```

Step 4: Launch Backend API Server (Port 4000)

```
npm run dev
# Listening at http://127.0.0.1:4000
```

Step 5: Launch Frontend Client Server (Port 5173)

```
cd ..\frontend
npm install
npm run dev
# Local: http://127.0.0.1:5173/
```

Step 6: Provision Staff or Recruiter Accounts (PowerShell CLI)

```
cd ..\backend
.\scripts\Provision-User.ps1 -Email officer@college.edu -Role STAFF
```

CAMPUSLINK ARCHITECTURAL SUMMARY

By fusing deterministic matching algorithms, transparent rubric-based sprint assessments, cryptographically grounded verification pipelines, and local-first relational storage in PostgreSQL, CAMPUSLINK provides colleges, students, and corporate recruiters with an honest, verifiable, and high-integrity Career Intelligence OS.